

Dobot Magician Robot System: ROS 1 (Noetic), OpenCV, pydobot2 & PyQt5 Full Guide

Autonomous Pick-and-Place, Multi-Color Segmentation, 3-Phase Mission Execution & Collision Avoidance (From Scratch)

Author / Architect: Thanadech
(Google Deepmind Pair-
Programming Assistant)

Operating System: Ubuntu Linux
20.04 LTS / ROS 1 Noetic
Ninjemys

Catkin Workspace: /root/Dobot-
Magician-Autonomous-Stacking-September
ROS1

Release Date:
September
2026

Guide Objective

This comprehensive documentation is tailored for students, engineers, and researchers starting with zero robotics or ROS 1 background. It explains every concept from the ground up: communication middleware, centralized master architecture, perspective computer vision, robot kinematics, asynchronous PyQt5 GUI design, 3-phase automated obstacle clearance, and goal collision avoidance.

Table of Contents

- Chapter 1: System Overview & Physical Field Geometry
- Chapter 2: ROS 1 Fundamentals from Scratch (Master, Nodes, Topics, Parameters, Launch)
- Chapter 3: Computer Vision Pipeline with OpenCV & ROS 1 Image Transport
- Chapter 4: Dobot Magician Control with pydobot2 & Hardware Controller Node
- Chapter 5: GUI Architecture with PyQt5 (Event Loop, Signals & Slots, Thread-Safe RosBridge)
- Chapter 6: 3-Phase Mission Execution & Goal Collision Avoidance System
- Chapter 7: UI Dashboard Walkthrough with Screenshots
- Chapter 8: Build, Execution & Troubleshooting Guide

Chapter 1: System Overview & Physical Field Geometry

1.1 Physical Layout & Coordinate Frame

The operational setup is configured on an international standard A4 sheet (210 x 297 mm) serving as the calibrated ground plane, featuring four core spatial landmarks:

1. **Dobot Magician Robot Base:** Mounted at field origin coordinates ($X = 105.0$ mm, $Y = 270.0$ mm).
2. **3x3 Pallet Grid:** An outer bounding footprint of 114.5 x 114.5 mm with a 35.0 mm cell pitch and 25.0 mm cell cutouts. The 8 outer grid cells host colored cubes (Red, Yellow, Green, Blue).
3. **Center Goal Cell [1, 1]:** The central drop target where selected cubes are vertically stacked (strictly locked to a 4-block maximum ceiling).
4. **4 Feeder Slots (Feeder 1 to 4):** Situated on the left perimeter strip ($Y \sim +79.3$ mm). Marked with green rings, these act as designated holding stations for obstacle cubes.

```
[4 Feeder Slots]      [3x3 Pallet Grid: 114.5 x 114.5 mm]
(Green Rims)
( #1 )                +-----+-----+-----+
                        | (0,0) | (0,1) | (0,2) |
( #2 )                +-----+-----+-----+
                        | (1,0) | (1,1) | (1,2) |
( #3 )                |      GOAL      |
                        +-----+-----+-----+
( #4 )                | (2,0) | (2,1) | (2,2) |
                        +-----+-----+-----+

                        [Dobot Magician Robot Base]
                        (105, 270) mm
```

1.2 Catkin Workspace Architecture

The Catkin workspace (`/root/Dobot-Magician-Autonomous-Stacking-R0S1`) separates concerns into modular ROS 1 packages:

- `dobot_v2` : Core vision processing node (`detection_node.py`), homography coordinate transforms, serial hardware driver (`dobot_driver.py`), controller node (`dobot_controller_node.py`), and 3-phase trajectory sequencer (`mission_executor.py`).
- `Dobot_UI` : Rich interactive PyQt5 dashboard providing live camera streaming, robot status telemetry, mission sequencing, and manual Cartesian jogging.
- `dobot_project` : Standalone single-node reference implementation for standalone experiments.

Chapter 2: ROS 1 Fundamentals from Scratch

2.1 What is ROS 1 Noetic?

ROS 1 (Robot Operating System 1 - Noetic Ninjemys) is a distributed robotics software middleware designed for Linux (Ubuntu 20.04 LTS). Unlike monolithic software systems, ROS 1 decomposes robotic tasks into independent executables (called **Nodes**) that communicate over a network or shared memory using standard message structures.

Centralized ROS Master (`roscore`): In ROS 1, a central name server known as the Master runs on port 11311. When nodes start up, they register their publishers, subscribers, and services with the Master via XML-RPC. Once discovered, nodes establish direct, high-throughput point-to-point connections over TCP/IP (TCPROS) or UDP (UDPROS) to stream data.

2.2 Core ROS 1 Primitives

1. Nodes (`rospy.init_node`)

A Node is an independent executable process responsible for a single task:

- `usb_cam` : Captures raw video frames from the USB webcam via V4L2 and publishes them to ROS.
- `detection_node` : Segments cubes in HSV space, tracks the pallet grid, and publishes metric 3D coordinates.
- `dobot_controller` : Communicates over serial with the Dobot arm, manages safety limits, and executes trajectories.
- `dobot_ui` : Pure PyQt5 operator dashboard providing real-time telemetry and user controls.

2. Topics & Publisher-Subscriber Pattern

Topics provide unidirectional, asynchronous streaming data channels. A Publisher broadcasts messages without knowing who is listening, and Subscribers trigger callback functions whenever a new message arrives.

Topic Name	Message Type	Publisher	Subscriber	Description
/usb_cam/image_raw	sensor_msgs/Image	usb_cam	detection_node	Raw uncompressed camera frame stream (640x480 @ 30 FPS, YUV420P via mmap).
/usb_cam/camera_info	sensor_msgs/CameraInfo	usb_cam	detection_node	Camera intrinsic calibration matrix and distortion coefficients.
/detected_objects	std_msgs/String (JSON)	detection_node	Dobot_UI , dobot_controller	Real-time metric coordinates (X, Y, Z), colors, group assignments, and lock status.
/detected_objects_image	sensor_msgs/Image	detection_node	rqt_image_view	Raw annotated image stream with bounding boxes, homography grid, and visual HUD.
/detected_objects_image/compressed	sensor_msgs/CompressedImage	detection_node	Dobot_UI	Low-latency JPEG compressed stream rendered inside PyQt5 dashboard.
/detected_objects_poses	geometry_msgs/PoseArray	detection_node	ROS Navigation / RViz	3D poses of detected cubes in Dobot base frame (dobot_base_link).
/grid_local_poses	geometry_msgs/PoseArray	detection_node	External Planners	3D poses of cubes relative to pallet grid origin (grid_origin).
/dobot_status	std_msgs/String (JSON)	dobot_controller	Dobot_UI	Telemetry: X, Y, Z, R Cartesian pose, suction pump status, mission phase, and alarms.

Topic Name	Message Type	Publisher	Subscriber	Description
/dobot_ui_cmd	std_msgs/String (JSON)	Dobot_UI	dobot_controller	Operator commands: start mission, jog axes, home robot, emergency stop.
/set_grid_corners	std_msgs/String (JSON)	calibrate_grid , UI	detection_node	Dynamic manual update of calibrated homography corners.

3. ROS 1 Parameter Server & Launch Files

ROS 1 manages parameters centrally on the Master. Nodes query private parameters using the tilde prefix (e.g. `rospy.get_param('~image_topic')`). XML launch files load parameter configurations directly into node private namespaces using `<rosparam command="load" file="..." />`. Unlike ROS 2 (which requires `/**: ros__parameters:` nesting), ROS 1 YAML configuration files must be flat key-value dictionaries.

Chapter 3: Computer Vision Pipeline with OpenCV

3.1 Camera Capture & V4L2 Video Streaming

On Linux, USB webcams interface via the Video4Linux2 (V4L2) kernel subsystem. The ROS 1 `usb_cam_node` captures frames directly from `/dev/video0` using memory-mapped I/O (`io_method: "mmap"`) and native YUYV uncompressed pixel format at 640 x 480 resolution and 30 FPS, delivering minimal latency without excessive CPU overhead.

3.2 Why HSV is Superior to RGB/BGR for Color Segmentation

Standard cameras output images in **BGR** (Blue, Green, Red) . In BGR, illumination changes (shadows, daylight shifts, overhead flickering) simultaneously alter all three color channels, making thresholding unstable.

In contrast, **HSV** (Hue, Saturation, Value) isolates chromaticity from illumination:

- **H (Hue [0..180])**: Encodes the pure spectral color independent of light intensity (e.g. Green is always $\sim 36..85$ regardless of room lighting; Red wraps at 0..10 and 170..180).
- **S (Saturation [0..255])**: Quantifies color purity, distinguishing vivid cube colors from white paper and highlights.
- **V (Value [0..255])**: Quantifies luminance, allowing dark shadows to be filtered out cleanly.

3.3 Contour Analysis & Geometric Filtering

Binary color masks produced via `cv2.inRange()` are cleaned using morphological opening (erosion followed by dilation). Contours are evaluated against strict geometric criteria to eliminate reflections and background marks:

- **Area Bounds**: Area must fall between 250 and 30,000 pixels.
- **Aspect Ratio**: Maximum bounding box aspect ratio ≤ 2.2 (cubes appear square or slightly rectangular when viewed from an angle).
- **Solidity**: Contour area divided by convex hull area ≥ 0.78 (solid polygons).
- **Circularity Filter**: Rejects printed green circular feeder rings on paper (circularity ≤ 0.84), ensuring only genuine 3D cubes are detected.

3.4 Pallet Grid Detection & Auto-Locking

The pallet frame is segmented using neutral grey thresholding (Value between 105 and 195, Saturation ≤ 50). The largest quadrilateral contour is fitted and sorted into canonical corner order: `P0` (Bottom-Right), `P1` (Bottom-Left), `P2` (Top-Left), `P3` (Top-Right) . Temporal Exponential Moving Average (EMA, $\alpha = 0.25$) stabilizes corner coordinates across frames. Once the pallet remains stable for 10 consecutive frames, it is locked automatically to eliminate jitter.

3.5 Metric Perspective Homography

Because the overhead camera views the table at an oblique angle, perspective keystoning distorts physical distances. We compute a 3x3 Homography transform matrix:

1. Detect the 4 outer corner coordinates of the grid in image pixel space (u, v).
2. Define target metric dimensions: a square 114.5 x 114.5 mm planar grid.
3. Calculate the matrix: `cv2.getPerspectiveTransform(src_pts, dst_pts)` .

4. Multiply any detected pixel centroid (u, v) by this homography matrix to obtain table-space millimeters (gx, gy), and map directly into Dobot robot base coordinates (rx, ry, rz).

3.6 High-Performance ROS 1 Image Serialization

To prevent Python 3 / OpenCV version mismatches (such as the notorious `KeyError: 16` occurring when newer OpenCV releases conflict with system `cv_bridge` binaries), `detection_node.py` features a dedicated, direct numpy buffer serializer:

```
def _cv2_to_imgmsg(self, cvim: np.ndarray, header=None, encoding: str = 'bgr8') -> Image:
    img_msg = Image()
    if header is not None:
        img_msg.header = header
    img_msg.height, img_msg.width = cvim.shape[0], cvim.shape[1]
    img_msg.encoding = encoding
    img_msg.step = cvim.shape[1] * (1 if cvim.ndim == 2 else cvim.shape[2])
    img_msg.data = cvim.tobytes()
    return img_msg
```

This eliminates all external binary C++ conversion overhead and ensures 100% reliable image streaming on both raw (`/detected_objects_image`) and compressed (`/detected_objects_image/compressed`) topics.

Chapter 4: Dobot Magician Control with pydobot2

4.1 Serial UART Protocol

The Dobot Magician communicates via a USB-to-UART serial interface at `115200 bps`. The protocol transmits structured binary packets: `Header (0xAA 0xAA) | Length | Command ID | Payload | Checksum`. The `dobot_driver.py` module encapsulates this protocol with auto-detecting serial port capabilities and hardware command queuing.

4.2 Cartesian Workspace Limits

Dobot Magician Working Envelope

- **X (mm)**: Forward/backward extension. Safe limits: 100.0 mm to 330.0 mm.
- **Y (mm)**: Lateral left/right rotation. Limits: -250.0 mm to +250.0 mm.
- **Z (mm)**: Vertical elevation. Limits: -60.0 mm to +160.0 mm.
- **R (deg)**: End-effector wrist orientation. Limits: -180.0° to +180.0°.

4.3 PTP Motion Modes: MOVJ vs MOVL

- **MOVJ_XYZ** (Joint Movement): All joints interpolate simultaneously. The arm swings in a smooth, high-speed arc. Used for open transit between field waypoints.
- **MOVL_XYZ** (Linear Movement): The end-effector moves strictly along a Cartesian straight line. Crucial for picking up cubes vertically (downward 90°) and rising clear of neighbors without lateral collisions.

4.4 End-Effector Control

The robot is equipped with a pneumatic suction cup actuated via an onboard vacuum pump. Settling dwell timers (0.35s suction dwell after grip, 0.25s release dwell) guarantee that vacuum pressure stabilizes completely before the arm executes vertical transit.

Chapter 5: GUI Architecture with PyQt5

5.1 Event Loop & Thread Decoupling via `QThread`

PyQt5 runs an internal **Event Loop** that processes user inputs and repaints the screen. Running ROS 1 subscription loops (`rospy.spin()`) on this main thread would cause the UI to freeze and stop responding.

The Solution: We isolate ROS 1 communication into a dedicated background worker thread, `RosBridge(QThread)` :

1. **Main UI Thread**: Listens for mouse clicks, manages layouts, and renders Qt widgets.
2. **Worker Thread (`RosBridge`)**: Subscribes to ROS 1 topics in the background. Incoming image frames and status JSONs are emitted across threads via thread-safe `pyqtSignal` instances (`image_received` , `status_received` , `detections_received` , `log_message`), keeping the GUI 100% responsive.

5.2 Pure PyQt5 Implementation

In ROS 1 Noetic (Python 3.8), PyQt6 is not officially supported and introduces syntax and typing incompatibilities. The entire dashboard is developed in pure PyQt5, using standard Qt enums (e.g. `Qt.AlignCenter`, `Qt.Horizontal`, `QFrame.Box`) and standard application execution (`app.exec_()`).

5.3 Linux Unicode & Emoji Support

Standard Ubuntu 20.04 minimal installations lack emoji glyph fonts, causing UI icons to appear as blank whitespace or tofu boxes. The system is provisioned with `fonts-noto-color-emoji` and `fonts-symbola`, and the global QSS stylesheet explicitly includes `'Noto Color Emoji'` in the font stack, ensuring vibrant, crisp icons (, , , , , , ) render flawlessly.

Chapter 6: 3-Phase Mission Execution & Collision Avoidance

6.1 3-Phase Automated Mission Workflow

3-Phase Mission Architecture

1. **Phase 1 (Clear Obstacles):** Non-goal cubes located in the grid are picked and cleared to **Feeder Slots 1..4** on the left perimeter.
2. **Phase 2 (Stack Goal):** Target cubes are picked in sequence (#1..#4) and vertically stacked at **Center Goal [1, 1]** up to a strict 4-block limit ($Z = 30.0, 55.0, 80.0, 105.0$ mm).
3. **Phase 3 (Restore Obstacles):** Once goal stacking finishes, all obstacle cubes in the feeders are returned to their original grid slots.

6.2 Goal Collision Avoidance System

When 4 cubes are stacked at Goal [1, 1], the tower reaches 130 mm tall. A straight-line trajectory between a feeder ($Y = +79$ mm) and a right-column cell ($Y = -35$ mm) would slice directly through the center goal, toppling the tower.

Intelligent Detour & Clearance Trajectory:

- A cylindrical Keepout Zone ($R = 38.0$ mm) is established around Goal (176.0, 0.0).
- `_point_to_segment_dist()` evaluates candidate straight trajectories against this keepout cylinder.
- If a collision is detected, `_plan_safe_transit()` inserts a perimeter corridor detour waypoint:
 - **North Corridor Bypass:** Detours around the top perimeter via $X = 222.0$ mm.
 - **South Corridor Bypass:** Detours around the bottom perimeter via $X = 132.0$ mm.
- **Elevated Vertical Clearance:** Whenever cubes exist on the goal, transit elevation is automatically increased to $Z_{\text{transit}} = \max(\text{hover}_z, 142.0 \text{ mm})$, safely clearing the 130 mm tower.

Chapter 7: UI Dashboard Walkthrough with Screenshots

7.1 Stacking Mission Tab

Primary dashboard for configuring the 3-phase mission, assigning goal and obstacle roles, and executing autonomous stacking:

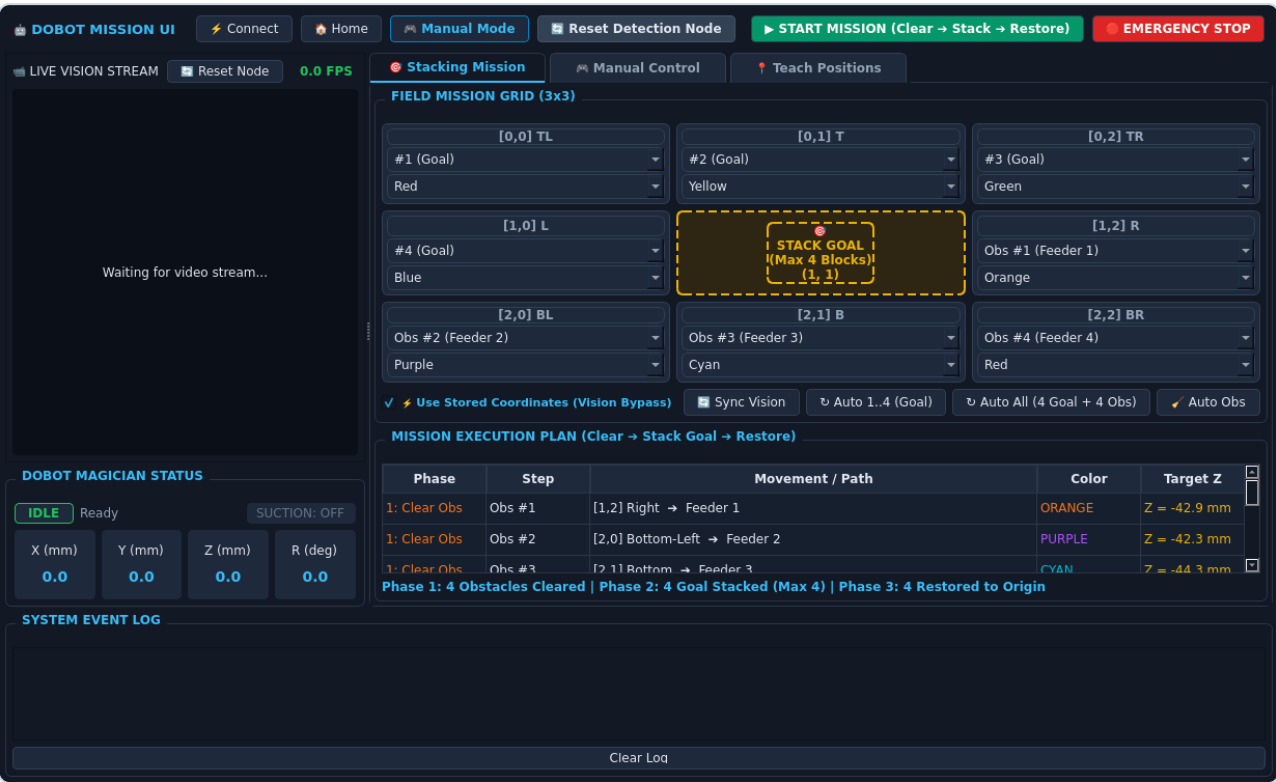


Figure 7.1: Interactive Stacking Mission Tab with Live Grid, Vision HUD, and Execution Plan.

- **Auto Buttons:** `Auto 1..4 (Goal)` automatically assigns colors to goal stack; `Auto All` plans all 8 blocks; `Auto Obs` auto-assigns obstacle clearance.
- **Execution Plan Table:** Displays every action step across all 3 phases in real-time with color-coded status badges.

7.2 Manual Control Tab

Direct Cartesian jogging, coordinate presets, and end-effector testing:

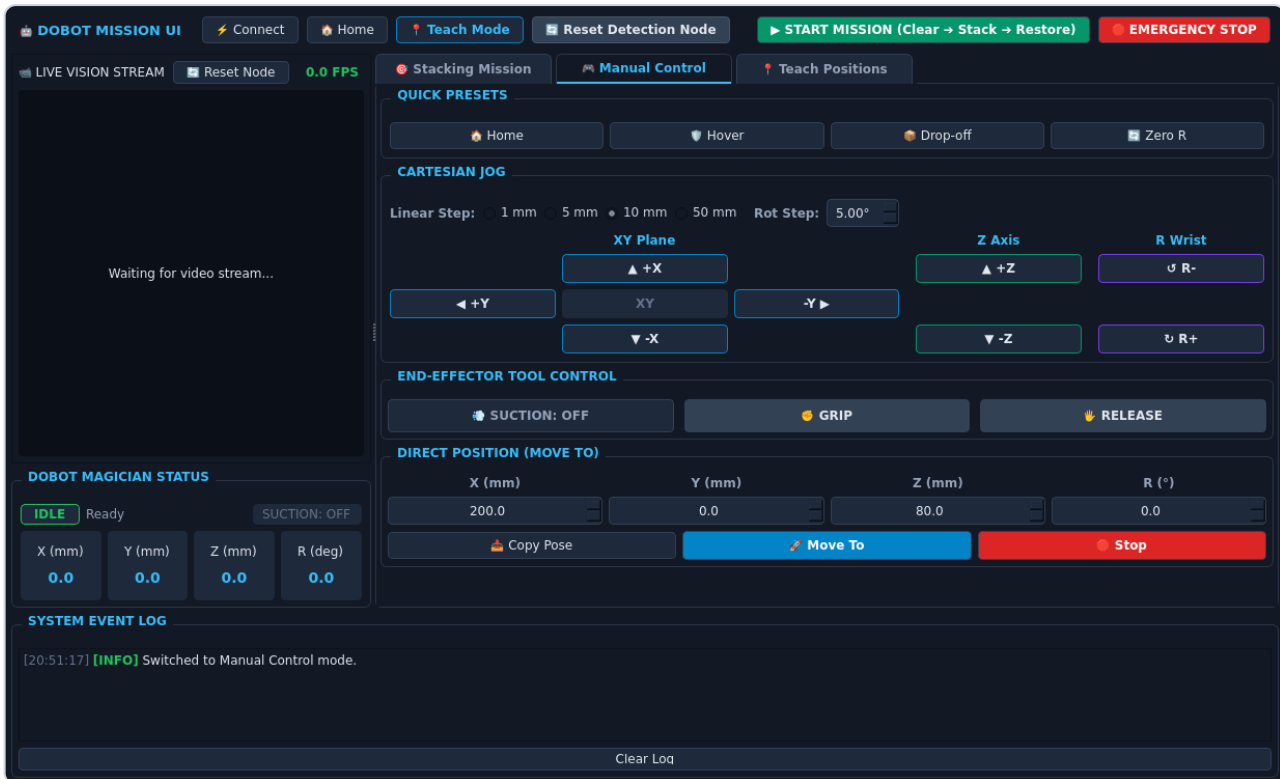


Figure 7.2: Manual Cartesian Jogging, Position Presets, and Pneumatic Suction Control.

- Quick Presets: Instant positioning to Home , Hover , Drop-off , and Zero R .
- Cartesian Jog: Directional buttons for X, Y, Z, R with step size selection (1, 5, 10, 50 mm).
- Tool Control: Pneumatic suction toggle (SUCTION: ON/OFF) and mechanical gripper controls.

7.3 Teach Positions Tab (Vision Bypass)

Enables manual position calibration and fail-safe operation when camera vision is occluded:

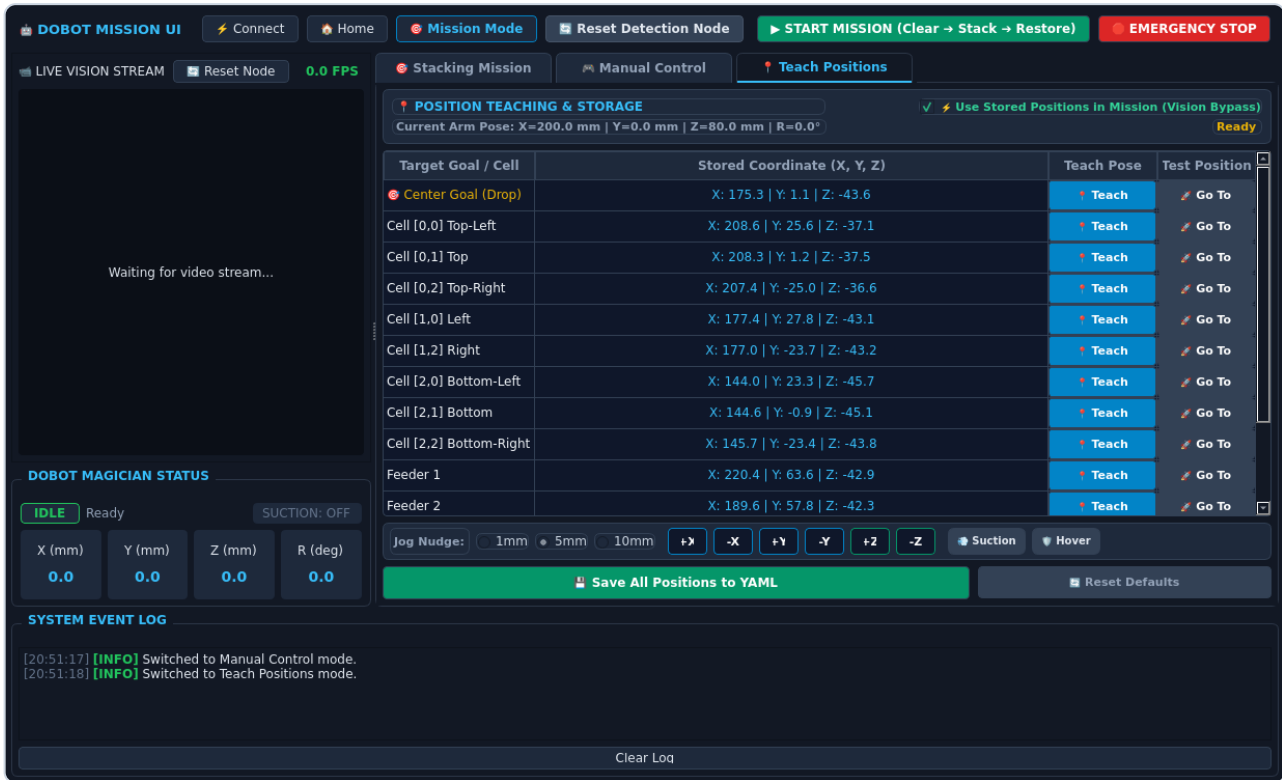


Figure 7.3: Position Teaching Matrix and YAML Configuration Persistence.

- Jog the robot tip to touch any cell or feeder and record coordinates with Teach .
- Save all calibrated positions to `dobot_ui.yaml` with Save All Positions to YAML .
- Vision Bypass Checkbox: Allows operating the stacking mission smoothly using taught coordinates if the camera is disconnected, lighting changes abruptly, or vision tracking fails.

Chapter 8: Build, Execution & Troubleshooting Guide

8.1 Build & Launch Commands

```
# 1. Source ROS 1 Noetic and build catkin workspace
cd /root/Dobot-Magician-Autonomous-Stacking-ROS1
source /opt/ros/noetic/setup.bash
catkin_make
source devel/setup.bash

# 2. Launch complete system (Camera + Detection + Controller + UI)
roslaunch dobot_v2 dobot_system.launch

# 3. Launch dedicated vision pipeline only
roslaunch dobot_v2 dobot_vision.launch

# 4. Visualize annotated video stream in rqt_image_view
rqt_image_view /detected_objects_image

# 5. Launch GUI in Mock Simulation Mode (no physical robot required)
roslaunch Dobot_UI dobot_ui.launch mock:=true
```

8.2 Troubleshooting & FAQ

USB Serial Permissions on Linux

If the controller reports `Permission denied: '/dev/ttyUSB0'` :

```
sudo usermod -a -G dialout $USER
sudo chmod 666 /dev/ttyUSB0
# Log out and log back in for group changes to take effect
```

Camera Video Node Reset

If the video stream freezes or reports `Device Busy` , click the  `Reset Detection Node` button on the top control bar of the GUI. This safely terminates hanging vision instances and restarts `detection_node` .

ROS 1 Topic Verifications

Verify live topic publication rates using standard ROS 1 command-line tools:

```
rostopic hz /detected_objects_image           # Should stream at ~20-30 Hz
rostopic hz /detected_objects_image/compressed# Should stream at ~20-30 Hz
rostopic echo /detected_objects                # Displays live JSON telemetry
```

Author: Thanadech (Google Deepmind Pair-Programming Assistant - Antigravity) • September 2026